

Bash Scripting Lab: Automating with Loops

Loops in Bash Scripting

Purpose of Loops in Bash

- Automates repetitive tasks without manual intervention.
- Allows execution of the same set of commands multiple times.
- Simplifies scripts by eliminating the need to duplicate code for repeated tasks.
- Enhances efficiency, especially when working with large data sets or files.

Types of Loops in Bash

- 1) **For Loop:** Iterates over a list of items (e.g., files, strings, numbers) and executes commands for each item.
- 2) **While Loop:** Executes commands as long as a specified condition is true, often used for operations with variable conditions.
- 3) **Until Loop:** Similar to the while loop but runs until a specified condition becomes true.
- 4) **Select Loop:** Useful for creating simple menus, allowing users to select from a list of options.

Common Use Cases for Loops

- **File Processing:** Perform actions on each file in a directory (e.g., renaming, moving, or analyzing files).
- **Data Iteration:** Process data line by line, such as reading and manipulating entries in a CSV or text file.
- **Automating Command Sequences:** Repeat commands across various systems or resources, such as multiple servers or directories.
- **Task Scheduling:** Repeatedly execute a task until certain conditions are met, useful for monitoring or checking resource availability.

Advantages of Using Loops in Bash

- **Code Efficiency:** Reduces code redundancy, leading to cleaner and more readable scripts.
- **Flexibility:** Easily adjust loops to handle different data sets or operational requirements.

The University of Lahore

- **Error Reduction:** Fewer lines of code reduce the potential for mistakes, as one loop can replace dozens of repetitive lines.
- **Scalability:** Scripts using loops can scale to process larger volumes of data without significant code changes.

For Loop in Bash Scripting

Purpose:

A `for` loop is used to execute a set of commands repeatedly for a specified number of times or for each item in a list or range.

Basic Syntax:

```
for variable in list
do
commands
done
```

- `variable` iterates over each item in `list`.
- The `do` keyword marks the start of the commands to be executed in each iteration, and `done` marks the end.

Example - Iterating Over a List:

```
for item in apple banana cherry
do
echo "Fruit: $item"
done
```

This loop will output each fruit name.

Example - Iterating Over a Range of Numbers:

```
for i in {1..5}
do
echo "Number: $i"
done
```

`{1..5}` generates a range from 1 to 5, printing each number.

The University of Lahore

Using C-style Syntax:

Bash also supports a C-style `for` loop syntax:

```
for (( i=1; i<=5; i++ ))
do
echo "Number: $i"
done
```

`(( i=1; i<=5; i++ ))` defines an initialization (`i=1`), condition (`i<=5`), and increment (`i++`), similar to C language loops.

Iterating Over Command Output:

`for` loops can iterate over the output of a command using command substitution:

```
for file in $(ls *.txt)
do
echo "Text file: $file"
done
```

Here, `$(ls *.txt)` lists all `.txt` files, and each filename is processed in the loop.

Using Wildcards with For Loops:

You can loop over files directly with wildcards:

```
for file in *.txt
do
echo "Processing $file"
done
```

Loop Control Statements:

break: Exit the loop prematurely.

continue: Skip the current iteration and move to the next.

```
for i in {1..5}
```

The University of Lahore

```
do
if [ $i -eq 3 ]; then
continue
fi
echo "Number: $i"
done
```

This loop skips printing the number 3.

Nested For Loops:

You can place `for` loops inside each other for multi-dimensional iteration:

```
for i in {1..3}
do
for j in {1..2}
do
echo "i: $i, j: $j"
done
done
```

While Loop in Bash Scripting**Purpose:**

A `while` loop in Bash is used to repeat a block of commands as long as a specified condition remains true.

Basic Syntax:

```
while [ condition ]
do
commands
done
```

- The `condition` is checked before each iteration. If true, the loop executes the `commands` inside; otherwise, it exits the loop.
- `do` marks the start of the commands to be executed, and `done` marks the end.

The University of Lahore**Example - Simple Counter:**

```
counter=1
while [ $counter -le 5 ]
do
echo "Counter: $counter"
((counter++))
done
```

This loop will print numbers from 1 to 5, incrementing `counter` by 1 each time.

Infinite Loops:

A `while` loop can be used to create an infinite loop by setting the condition to always be true:

```
while true
do
echo "This will loop forever"
done
```

Press `Ctrl+C` to stop an infinite loop in the terminal.

Using Commands in the Condition:

You can use command outputs or conditions directly in the loop's condition:

```
while ping -c 1 google.com &> /dev/null
do
echo "Google is reachable"
sleep 5
done
```

This loop will repeatedly ping Google every 5 seconds until it becomes unreachable.

Example - Reading User Input:

A `while` loop can be used to keep asking for user input until a certain condition is met:

The University of Lahore

```
while true
do
read -p "Enter a number (0 to exit): " num
[ $num -eq 0 ] && break
echo "You entered: $num"
done
```

This loop stops if the user enters 0.

File Processing:

`while` loops are often used for reading files line by line:

```
while read line
do
echo "Line: $line"
done < file.txt
```

This reads each line in `file.txt` and outputs it to the screen.

Loop Control Statements:

break: Exit the loop immediately.

continue: Skip the current iteration and proceed to the next.

```
counter=1
while [ $counter -le 5 ]
do
if [ $counter -eq 3 ]; then
((counter++))
continue
fi
echo "Counter: $counter"
((counter++))
done
```

This loop skips printing the number 3.

The University of Lahore**Example - Using Multiple Conditions:**

You can combine multiple conditions in a `while` loop:

```
counter=1
while [ $counter -le 10 ] && [ $counter -ne 8 ]
do
echo "Counter: $counter"
((counter++))
done
```

This loop stops when `counter` reaches 8.

Common Use Cases:

- Waiting for a process or service to start or stop.
- Repeating commands until a particular status or condition changes.
- Reading files or user input continuously.
- Running background jobs until a specific condition is met.

Until Loop in Bash Scripting**Purpose:**

An `until` loop in Bash is used to execute a block of commands until a specified condition becomes true (the opposite of a `while` loop, which runs as long as the condition is true).

Basic Syntax:

```
until [ condition ]
do
commands
done
```

- The `condition` is checked before each iteration. The loop continues as long as the condition is **false**. Once the condition becomes true, the loop exits.

The University of Lahore**Example - Simple Counter:**

```
counter=1
until [ $counter -gt 5 ]
do
echo "Counter: $counter"
((counter++))
done
```

- This loop will print numbers from 1 to 5, incrementing `counter` by 1 each time. It stops when `counter` becomes greater than 5.

Using Commands in the Condition:

You can use command outputs or other conditions directly in the `until` loop condition:

```
until ping -c 1 google.com &> /dev/null
do
echo "Waiting for network..."
sleep 5
done
```

- This loop will continue to check network connectivity to Google every 5 seconds until it is reachable.

Example - User Input:

An `until` loop can keep prompting the user until a specific input is provided:

```
until [ "$password" == "secret" ]
do
read -sp "Enter password: " password
echo
done
echo "Password accepted"
```

The loop stops when the user enters the correct password, "secret".

The University of Lahore**File Processing:**

An `until` loop can be used for file operations, such as waiting until a file exists:

```
until [ -f /path/to/file ]
do
echo "Waiting for file to be created..."
sleep 2
done
echo "File found!"
```

- This example waits until `/path/to/file` exists before proceeding.

Loop Control Statements:

break: Exit the loop immediately.

continue: Skip the current iteration and proceed to the next.

```
counter=1
until [ $counter -gt 5 ]
do
if [ $counter -eq 3 ]; then
((counter++))
continue
fi
echo "Counter: $counter"
((counter++))
done
```

- This example skips printing 3.

Using Multiple Conditions:

Multiple conditions can be combined using logical operators within the loop condition:

```
counter=1
until [ $counter -gt 10 ] || [ $counter -eq 8 ]
```

The University of Lahore

```
do
echo "Counter: $counter"
((counter++))
done
```

- This loop stops when `counter` is greater than 10 or equal to 8.

Infinite Until Loop:

An `until` loop can be set up to run indefinitely if the condition never becomes true:

```
until false
do
echo "This will run forever"
done
```

- Press `Ctrl+C` to stop an infinite loop.

Common Use Cases:

- Waiting for a condition or status to change (e.g., a service or file becoming available).
- Prompting user input until valid data is entered.
- Running checks until specific criteria are met, such as network connectivity or completion of a process.

Select Loop in Bash Scripting**Purpose:**

The `select` loop provides a simple way to create a menu from which a user can select options. It automatically handles the user input and displays a numbered list of options.

Basic Syntax:

```
select variable in option1 option2 option3 ...
do
commands
done
```

The University of Lahore

- The `variable` will hold the value of the selected option, and `commands` will execute based on the user's choice.

Example - Simple Menu:

```
#!/bin/bash

PS3="Please select an option:
" # Prompt for selection
select fruit in Apple Banana
Cherry Quit
do
case $fruit in
Apple)
echo "You selected Apple."
;;
Banana)
echo "You selected Banana."
;;
Cherry)
echo "You selected Cherry."
;;
Quit)
echo "Exiting..."
break # Exit the loop
*)
echo "Invalid option. Please
try again."
;;
esac
done
```

PS3 Variable:

- The `PS3` variable defines the prompt message that appears before the user's input.
- It can be customized to guide the user on what to do.

Automatic Numbering:

The `select` loop automatically numbers the options for the user, making it easy to select an option by entering the corresponding number.

Example - Menu with Loop Control:

```
#!/bin/bash

PS3="Please select your favorite
programming language: "
select language in Python Java C++
Exit
do
case $language in
Python)
echo "You selected Python.>";;
Java)
echo "You selected Java.>";;
C++)
echo "You selected C++.";;
Exit)
echo "Goodbye!"
break;;
*)
echo "Invalid selection. Try again."
;;
esac
done
```

The University of Lahore**Invalid Input Handling:**

Using the `case` statement, you can handle invalid selections and prompt the user to try again.

Infinite Loop Until Valid Selection:

The `select` loop continues to display the options until the user makes a valid selection or chooses to exit.

Accessing the Index of the Selection:

You can also access the index of the selection using the special variable `REPLY`, which contains the user's input:

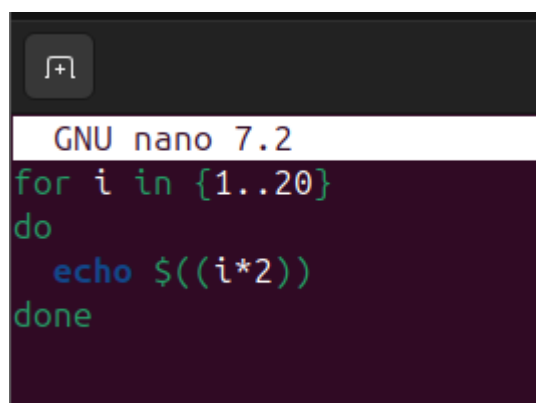
```
select option in "Option 1" "Option 2" "Option 3"
do
echo "You selected option number $REPLY which is $option."
break # To exit after the first selection
done
```

Common Use Cases:

- Creating interactive scripts that require user input for configuration or choices.
- Building menus for scripts that perform different tasks based on user selection.
- Providing a user-friendly interface for command-line applications.

Practice Questions

- 1) Write a `for` loop that prints the first 10 even numbers (2, 4, 6, ..., 20).



```
GNU nano 7.2
for i in {1..20}
do
    echo $((i*2))
done
```

The University of Lahore

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q1.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q1.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q1.sh
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
```

- 2) Create a `while` loop that counts down from 10 to 1 and prints each number. After reaching 1, print "Liftoff!".

```
GNU nano 7.2
i=10
while [ $i -ge 1 ]
do
    echo $i
    i=$((i-1))
done
echo "Liftoff!"
```

The University of Lahore

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q2.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q2.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q2.sh
10
9
8
7
6
5
4
3
2
1
Liftoff!
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

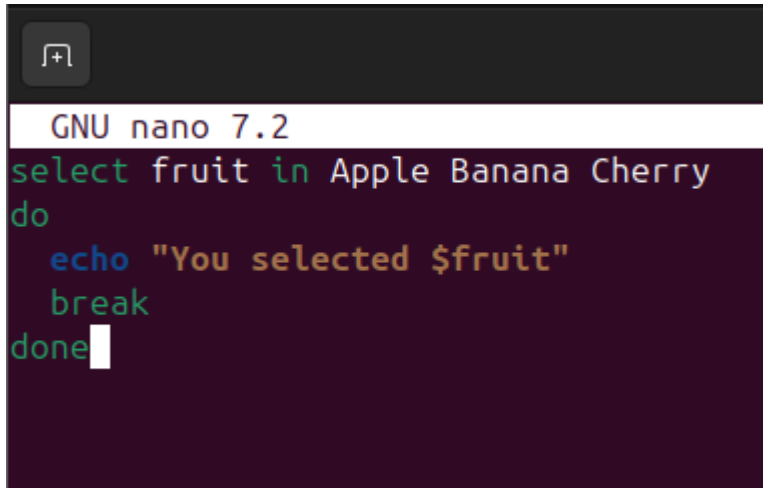
- 3) Write an `until` loop that waits for a file named `ready.txt` to exist. Once it exists, print "File is ready!".

```
GNU nano 7.2
until [ -f "ready.txt" ]
do
    sleep 1
done
echo "File is ready!"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q3.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q3.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q3.sh
File is ready!
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 4) Use a `select` loop to present the user with a menu of three fruits: Apple, Banana, and Cherry. Print the user's selection.

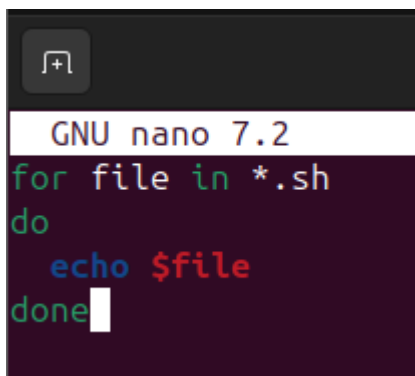
The University of Lahore



```
GNU nano 7.2
select fruit in Apple Banana Cherry
do
    echo "You selected $fruit"
    break
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q4.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q4.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q4.sh
1) Apple
2) Banana
3) Cherry
#? 2
You selected Banana
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 5) Write a `for` loop that iterates over all `.sh` files in the current directory and prints their names.



```
GNU nano 7.2
for file in *.sh
do
    echo $file
done
```

The University of Lahore

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q5.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q5.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q5.sh
check_file.sh
demo.sh
Q1.sh
Q2.sh
Q3.sh
Q4.sh
Q5.sh
rename_logs.sh
script16.sh
task2.sh
task3.sh
task4.sh
task5.sh
task6.sh
task7.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 6) Create a script using a `while` loop that prompts the user to enter a number and continues until the user enters a negative number.

```
GNU nano 7.2
num=0
while [ $num -ge 0 ]
do
    read -p "Enter a number: " num
done
echo "Negative number entered. Stopped."
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q6.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q6.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q6.sh
Enter a number: 12
Enter a number: 10
Enter a number: -9
Negative number entered. Stopped.
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 7) Write a nested `for` loop that prints a multiplication table for numbers 1 to 3.

The University of Lahore



```
GNU nano 7.2
for i in {1..3}
do
    for j in {1..3}
    do
        echo "$i x $j = $((i*j))"
    done
done
```

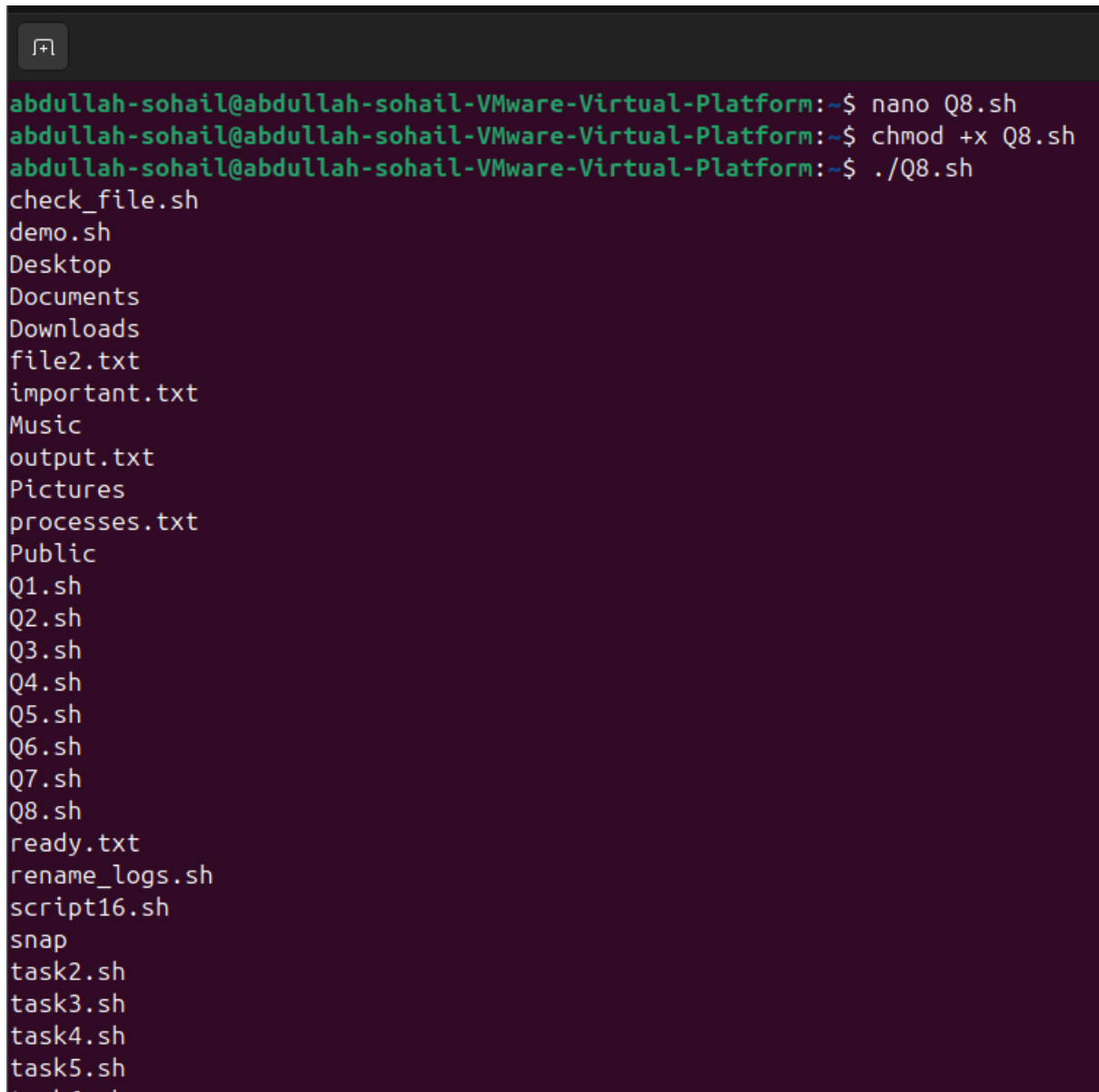
```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q7.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q7.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q7.sh
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 8) Use a `for` loop to print all files in a directory, skipping hidden files (those starting with a dot).



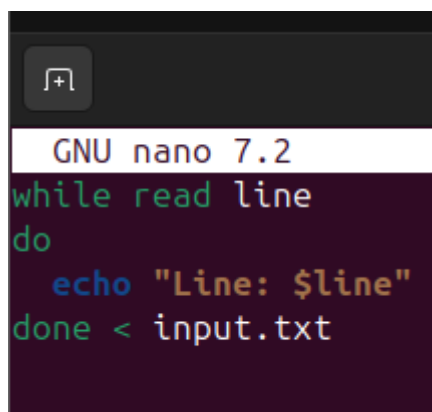
```
GNU nano 7.2
for file in *
do
    echo $file
done
```

The University of Lahore



```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q8.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q8.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q8.sh
check_file.sh
demo.sh
Desktop
Documents
Downloads
file2.txt
important.txt
Music
output.txt
Pictures
processes.txt
Public
Q1.sh
Q2.sh
Q3.sh
Q4.sh
Q5.sh
Q6.sh
Q7.sh
Q8.sh
ready.txt
rename_logs.sh
script16.sh
snap
task2.sh
task3.sh
task4.sh
task5.sh
```

- 9) Create a script that uses a `while` loop to read each line of a text file named `input.txt` and prints each line prefixed with "Line: ".



```
GNU nano 7.2
while read line
do
    echo "Line: $line"
done < input.txt
```

[illegible]

The University of Lahore

- 11) Modify the previous script to use a `while` loop instead of `until`, printing "Waiting for condition..." until `condition` is true.

```
GNU nano 7.2
condition=false

while ! $condition
do
    echo "Waiting for condition..."
    sleep 1
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q11.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q11.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q11.sh
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
Waiting for condition...
```

- 12) Create a script using a `for` loop to read a list of names (Alice, Bob, Charlie) and greet each person with "Hello, [name]!".

The University of Lahore

```
GNU nano 7.2
for name in Alice Bob Charlie
do
    echo "Hello, $name!"
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q12.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q12.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q12.sh
Hello, Alice!
Hello, Bob!
Hello, Charlie!
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 13) Write a script that uses a `select` loop to let the user choose a color from a list, and display a message based on their choice.

```
GNU nano 7.2
select color in Red Blue Green
do
    case $color in
        Red) echo "You chose Red";;
        Blue) echo "You chose Blue";;
        Green) echo "You chose Green";;
    esac
    break
done
```

The University of Lahore

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q13.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q13.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q13.sh
1) Red
2) Blue
3) Green
#? 1
You chose Red
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

14) Create a `for` loop that calculates the factorial of a number (e.g., 5) and prints the result.

```
GNU nano 7.2
num=5
fact=1

for i in $(seq 1 $num)
do
    fact=$((fact*i))
done

echo "Factorial of $num is $fact"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q14.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q14.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q14.sh
Factorial of 5 is 120
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

15) Write a script that uses a `while` loop to continuously check if a particular service (e.g., `httpd`) is running. If it's not running, print "Service is down!" and wait for 5 seconds before checking again.

The University of Lahore

```
GNU nano 7.2
while true
do
    if ! systemctl is-active --quiet httpd
    then
        echo "Service is down!"
    fi
    sleep 5
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q15.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q15.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q15.sh
Service is down!
Service is down!
Service is down!
Service is down!
Service is down!
Service is down!
Service is down!
Service is down!
Service is down!
Service is down!
Service is down!
```

Automation with Files

- 16) Write a Bash script that counts the number of lines in all `.txt` files in the current directory and prints the total count.

```
GNU nano 7.2
total=0

for file in *.txt
do
    lines=$(wc -l < "$file")
    total=$((total + lines))
done

echo "Total lines: $total"
```

The University of Lahore

```
ab
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q16.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q16.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q16.sh
Total lines: 1316
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 17) Create a script that renames all `.log` files in the current directory by appending the current date to their filenames.

```
GNU nano 7.2
for file in *.log
do
    mv "$file" "${file}_${date +%Y-%m-%d}"
done
```

```
abdu
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q17.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q17.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q17.sh
mv: cannot stat '*.log': No such file or directory
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 18) Write a script that checks for the existence of a file named `important.txt`. If it does not exist, create it and write the current date and time into it.

```
GNU nano 7.2
if [ ! -f important.txt ]
then
    date > important.txt
    echo "File created with date & time"
else
    echo "File already exists"
fi
```

The University of Lahore

```
File already exists
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q18.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q18.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q18.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q18.sh
File already exists
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 19) Create a script that searches for the string "ERROR" in all *.log files and writes the filenames containing the string to error_files.txt.

```
GNU nano 7.2
> error_files.txt # clear file

for file in *.log
do
    if grep -q "ERROR" "$file"
    then
        echo "$file" >> error_files.txt
    fi
done

echo "Done. Check error_files.txt"
```

```
File already exists
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q19.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q19.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q19.sh
grep: *.log: No such file or directory
Done. Check error_files.txt
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 20) Write a script that lists all the files in the current directory, and if a file is larger than 1MB, moves it to a folder named large_files.

The University of Lahore

```
GNU nano 7.2
mkdir -p large_files

for file in *
do
    if [ -f "$file" ] && [ $(stat -c%s "$file") -gt 1048576 ]
    then
        mv "$file" large_files/
    fi
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q20.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q20.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q20.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

21) Write a script that creates a backup of all `.conf` files by copying them to a backup directory, appending the current date to each filename.

```
GNU nano 7.2
mkdir -p backup

for file in *.conf
do
    cp "$file" "backup/${file}_${date +%Y-%m-%d}"
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q21.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q21.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q21.sh
```

22) Create a script that deletes all empty files in the current directory.

The University of Lahore

```
GNU nano 7.2
find . -type f -empty -delete
echo "Empty files deleted"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q22.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q22.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q22.sh
Empty files deleted
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

23) Write a script that finds all `.jpg` files and compresses them using the `gzip` command.

```
GNU nano 7.2
for file in *.jpg
do
    gzip "$file"
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q23.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q23.sh
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q23.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

24) Create a script that finds duplicate files (by content) in the current directory and moves them to a folder named `duplicates`.

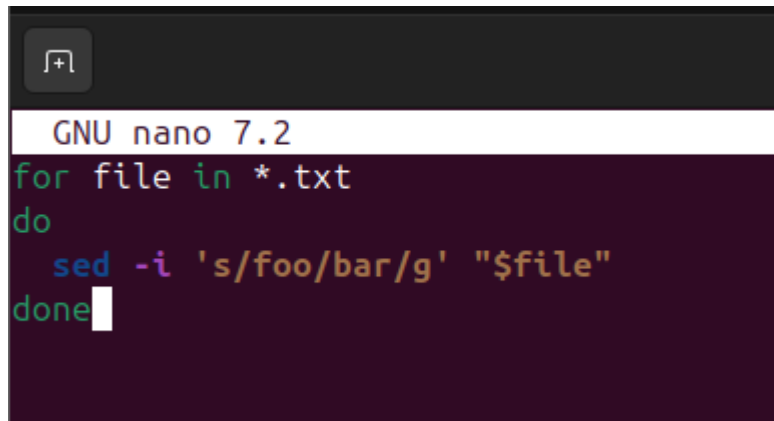
```
GNU nano 7.2
mkdir -p duplicates

for file1 in *
do
    for file2 in *
    do
        if [ "$file1" != "$file2" ] && cmp -s "$file1" "$file2"
        then
            mv "$file2" duplicates/
        fi
    done
done
```

[illegible]

25) Write a script that replaces all occurrences of the word "foo" with "bar" in all `.txt` files and saves the changes.

The University of Lahore

A terminal window with a dark background. The title bar shows a window icon and a plus sign. The text inside the terminal is: GNU nano 7.2, followed by a for loop: for file in *.txt, then do, then an indented sed command: sed -i 's/foo/bar/g' "\$file", then done, and a cursor at the end of the line.

```
GNU nano 7.2
for file in *.txt
do
    sed -i 's/foo/bar/g' "$file"
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano Q25.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x Q25.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./Q25.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```